4COSC006C: Software Development I – Coursework specification (2025/26)	
Module leader	Guhanathan Poravi
Weighting:	50%
Qualifying mark:	30%
Description:	Coursework
Learning Outcomes Covered in this Assignment:	The coursework rationale is: **LO1** Analyse specific problems and design their solutions by applying appropriate algorithmic techniques; **LO2** Apply programming concepts to implement solutions in the taught programming language; **LO3** Implement and manipulate simple data structures; **LO4** Use an integrated development environment to create programs to satisfy a simple specification.
Handed Out:	Friday 06th Mar 2026
Due Date:	**Coursework Deadline**: Monday 6th Apr 2026 **BEFORE** 1:00 pm
Expected deliverables:	**Create ZIP File** • Select all required files • Create ZIP (not RAR or other formats) • Rename to StudentUoWID.zip
Method of Submission:	Submitted online via Blackboard
Type of Feedback and Due Date:	Feedback during viva and written feedback and marks by 1st of Jun 2026

Assessment regulations

Refer to the following for clarification on what constitutes plagiarism, collusion and penalties for late submissions.

This is an individual coursework. You should not share your coursework or parts of your coursework with another student as this can cause you both to receive an allegation of collusion:

https://www.westminster.ac.uk/current-students/guides-and-policies/academic-matters/academic-misconduct/collusion

Clarification on what constitutes plagiarism:
https://www.westminster.ac.uk/current-students/guides-and-policies/academic-matters/academic-misconduct/plagiarism

Penalty for Late Submission

If you submit your coursework late but within 24 hours or one working day of the specified deadline, 10 marks will be deducted from the final mark, as a penalty for late submission, except for work which obtains a mark in the range 40 – 49%, in which case the mark will be capped at the pass mark (40%). If you submit your coursework more than 24 hours or more than one working day after the specified deadline you will be given a mark of zero for the work in question unless a claim of Mitigating Circumstances has been submitted and accepted as valid.

It is recognised that on occasion, illness or a personal crisis can mean that you fail to submit a piece of work on time. In such cases you must inform the Campus Office in writing on a mitigating circumstances form, giving the reason for your late or non-submission. You must provide relevant documentary evidence with the form. This information will be reported to the relevant Assessment Board that will decide whether the mark of zero shall stand. For more detailed information regarding University Assessment Regulations, please refer to the following

website: https://www.westminster.ac.uk/current-students/guides-and-policies/academic-matters/academic-regulations

VIVA VOCE EXAMINATION

The viva voce examination is a **standard and essential academic practice** used in higher education institutions worldwide to verify the authenticity of student work and ensure academic integrity. This oral examination provides students with the opportunity to **demonstrate their understanding and ownership** of their submitted coursework in a fair and transparent manner. The viva is to validate that the work submitted genuinely reflects the student's own effort, knowledge, and capabilities. Students who have completed the coursework independently and understand their own code will find the viva a straightforward opportunity to showcase their learning and should have no difficulty in demonstrating their competence.

The University is committed to maintaining the highest standards of academic integrity and ensuring that all students are assessed fairly and equitably. The viva examination serves to **protect honest students** by ensuring that all submissions are evaluated on a level playing field, where marks are awarded based on genuine understanding and authentic individual work. This process benefits students who have worked diligently by ensuring their achievements are not undermined by academic misconduct from others. The assessment criteria for the viva are transparent, objective, and consistently applied to all students. Any reduction in marks following a viva will only occur when a student is **unable to demonstrate sufficient understanding** of their own submitted work, which raises legitimate concerns about the work's authenticity.

Rationale for Viva Voce Examination: In the current academic landscape, where advanced AI tools and code generation technologies are readily available, it has become increasingly important to verify that students genuinely understand and can explain the code they submit. The viva voce examination addresses this challenge by providing a direct assessment of each student's comprehension, problem-solving ability, and authentic engagement with their coursework. This practice ensures that grades accurately reflect individual competence rather than the capabilities of automated tools, thereby maintaining the integrity and value of academic qualifications.

Assignment Title: Digital Recipe Book Manager

Assignment Context

You are tasked with developing a **Digital Recipe Book Manager** - a command-line application that allows users to create, store, search, and manage cooking recipes. This system will help users organize their favorite recipes, track ingredients, categorize meals, and maintain their culinary collection.

The system must handle recipe data, validate user inputs, store information persistently, and provide intelligent search and filtering capabilities.

Submission Structure

Task	Description	Weight	Type
Task 1	Recipe Input & Validation	**20%**	
Task 2	Recipe Storage & Display	**20%**	
Task 3	File Management & Persistence	**25%**	
Task 4	Search & Advanced Features	**35%**	
Total		**100%**	

TASK 1 – Recipe Input & Validation (20%)

Objective

Implement robust input validation for recipe components with comprehensive error handling.

Requirements

1.1 Recipe Name Validation

- Must be between **3 and 50 characters**
- Must contain at least one letter
- Cannot be empty or contain only spaces
- Must reject special characters except: hyphen (-), apostrophe ('), and space
- Examples of valid names: "Grandma's Pie", "Chicken Stir-Fry"

1.2 Ingredient Input

- Each recipe requires **at least 3 ingredients, maximum 20**
- Each ingredient must have:
    - **Name** (3-30 characters)
    - **Quantity** (positive number, integer or decimal)
    - **Unit** (must be from allowed list: g, kg, ml, l, cup, tbsp, tsp, piece)
- Example valid ingredient: "Flour, 250, g"
- Must handle duplicate ingredient names by prompting user to confirm or reject

1.3 Cooking Time Validation

- Format: HH:MM (hours and minutes)
- Must be between 00:05 and 12:00
- Both hours and minutes must be valid (minutes: 0-59)
- Must reject: "25:00", "10:75", "abc"

1.4 Category Selection

- User must choose from predefined categories:
    - BREAKFAST
    - LUNCH

    - DINNER
    - DESSERT
    - SNACK
    - BEVERAGE
- Input should be case-insensitive
- Must re-prompt until valid category is entered

Required Behaviour

- Create a function validate_recipe_input() that collects all components
- Implement continuous prompting until valid input is provided
- Display **clear, specific error messages** for each type of validation failure
- Upon successful validation, display formatted summary:

Recipe Validated Successfully

Name: Chocolate Chip Cookies

Ingredients: 8

Category: DESSERT

Cooking Time: 00:45

Marking Criteria (20%)

Band	**Criteria**
0-30 (Super Lenient)	Basic input collection attempted; validation partially implemented; some error messages present; program runs but accepts invalid data
31-40 (Lenient)	Most validation rules implemented; handles common cases; error messages mostly clear; minor logical gaps in edge cases
41-69	All validation rules correctly implemented; robust error handling; clear user feedback; well-structured validation functions
70+ (Very Hard)	Flawless validation with all edge cases handled; elegant code structure; exceptional user experience; reusable validation functions; comprehensive error prevention

TASK 2 – Recipe Storage & Display (20%)

Objective

Design and implement an in-memory data structure system for storing multiple recipes with intelligent display capabilities.

Requirements

2.1 Data Structure Design

Design a **nested data structure** to store multiple recipes. Must use:

- **Dictionary** as the primary container (key: recipe ID)
- **Lists** for storing ingredients
- **Tuples** for immutable ingredient details (name, quantity, unit)
- **Set** for storing unique tags/categories (explained below)

Suggested structure:

```
recipes = {
   "RCP001": {
      "name": "Pasta Carbonara",
      "ingredients": [("Pasta", 200, "g"), ("Eggs", 2, "piece")],
      "time": "00:30",
      "category": "LUNCH",
      "tags": {"italian", "quick", "comfort-food"}
   }
}
```

2.2 Recipe ID Generation

- Auto-generate unique IDs in format: RCP + 3-digit number
- Start from RCP001
- Increment for each new recipe
- Must ensure no duplicate IDs

2.3 Multiple Recipe Entry

- Allow user to add **at least 3 different recipes**
- After each recipe, ask: "Add another recipe? (yes/no)"
- Store all recipes in the data structure
- Maintain count of total recipes added

2.4 Display Capabilities

Implement **three display functions**:

a) Display Single Recipe (Detailed)

```
==========================================
Recipe ID: RCP001
Name: Pasta Carbonara
Category: LUNCH
Cooking Time: 00:30
----------------------------------------
Ingredients:
  1. Pasta - 200 g
  2. Eggs - 2 piece
  3. Bacon - 150 g
----------------------------------------
Tags: italian, quick, comfort-food
==========================================
```

b) Display All Recipes (Summary)

```
Total Recipes: 5
----------------------------------------
RCP001 | Pasta Carbonara | LUNCH | 00:30
RCP002 | Chocolate Cake | DESSERT | 01:15
RCP003 | Green Smoothie | BEVERAGE | 00:05
...
```

c) Display by Category

- Prompt user to select category
- Display only recipes matching that category
- Show count: "Found 3 DESSERT recipes"

2.5 Basic Recipe Operations

Implement functions for:

- **Add recipe** to collection
- **View specific recipe** by ID
- **List all recipe IDs** with names
- **Count recipes** per category

Required Features

- Menu-driven interface with numbered options
- Input validation for menu choices
- Graceful handling of empty recipe book
- Clear feedback messages for all operations

Marking Criteria (20%)

Band	**Criteria**
0-30	Basic dictionary/list usage; stores some recipe data; display functions partially work; structure is unclear
31-40	Uses appropriate data structures; most storage/retrieval works; display functions mostly correct; some organizational issues
41-69	Correct use of all required data structures; clean implementation; all display functions work properly; well-organized code
70+	Optimal data structure design; elegant and efficient code; exceptional display formatting; perfect abstraction through functions; scalable design

TASK 3 – File Management & Persistence (25%)

Objective

Implement basic file handling functionality to save and load recipes, ensuring that data persists between program runs.

3.1 Save Recipes to File

Requirements

Implement functionality to save all stored recipes to a text file named:

recipes.txt

The file must use a structured and readable format that allows the data to be loaded back into the program later.

Suggested File Structure

```
===RECIPE===
ID:RCP001
NAME:Pasta Carbonara
CATEGORY:LUNCH
TIME:00:30
INGREDIENTS:Pasta,200,g|Eggs,2,piece|Bacon,150,g
TAGS:italian,quick,comfort-food
===END===
```

Functional Requirements

- Saving must be triggered through a user menu option
- Display confirmation after saving:

  "Recipes saved successfully to recipes.txt"

- Ensure all current recipes are written correctly to the file
- Ensure previously stored recipes are not lost during the save process
- Use clear and consistent formatting so the file can be reloaded accurately

Error Handling Expectations (Basic Level)

Students should:

- Handle situations where the file cannot be written in a simple and user-friendly way
- Display a clear message if saving fails
- Ensure program does not crash due to file write issues

3.2 Load Recipes from File

Requirements

Implement functionality to load recipes from **recipes.txt** when the program starts (if the file exists).

The program must:

- Read the structured file
- Parse recipe data correctly
- Reconstruct the in-memory data structure

Functional Requirements

- If file exists → load recipes automatically at program start
- If file does not exist → display:

  "No saved recipes found. Starting fresh."

- Validate loaded data format before adding into program
- If formatting issues are detected → display clear message and continue with empty recipe list
- Display count after successful loading:

  "Loaded X recipes successfully"

Error Handling Expectations (Basic Level)

Students should handle:

- Missing file
- Empty file

3.3 Save Reminder / Auto-Save Feature

Implement **one** of the following options:

Option A: Auto-save after every recipe addition

Option B: Prompt user to save before exiting

Option C: Auto-save after a fixed number of additions

The program must clearly inform the user when saving occurs.

Example:
"Auto-saving recipes..."
"All recipes saved successfully."

3.4 Export Recipe (Extension within this Task)

Create a function to export a single selected recipe into a separate formatted text file.

Requirements

- Filename format: recipe_<name>.txt
- Must be human-readable
- Clearly structured output

Example Output
RECIPE: Pasta Carbonara

==========

Category: LUNCH

Cooking Time: 30 minutes

INGREDIENTS:

- Pasta: 200 g
- Eggs: 2 piece
- Bacon: 150 g

TAGS: italian, quick, comfort-food

==========

3.5 File Operation Menu Integration

Extend the main menu to include file operations:

1. Add Recipe
2. View Recipes
3. Search Recipes
4. Save to File
5. Load from File
6. Export Recipe
7. Exit

Menu input must be validated and user-friendly.

Error Handling Scope for This Module

Students are expected to handle basic and commonly occurring issues:

- File not found
- Empty file
- Basic formatting errors in saved data
- Invalid or partial recipe entries

Marking Criteria (25%)

Band	Criteria
0–30	Basic file writing attempted; file reading partially works; inconsistent structure; limited validation; program may crash in some cases
31–40	Save and load functional for normal cases; basic validation present; handles missing file; file format mostly consistent
41–69	Reliable save/load implementation; correct structured format; clear user messages; maintains data integrity; clean and modular code
70+	Well-structured persistence design; efficient parsing; strong validation of loaded data; seamless integration with menu system; export feature implemented clearly and correctly

TASK 4 – Search & Advanced Features (35%)

Objective

Implement intelligent search, filtering, and advanced recipe management features demonstrating mastery of programming concepts.

4.1 Ingredient-Based Search (10%)

Implement a search function that finds recipes containing specific ingredients.

Requirements:

- User enters ingredient name (partial match allowed)
- Search is **case-insensitive**
- Returns all matching recipes
- Display format:

Found 3 recipes containing "chicken":

RCP004 | Chicken Curry | DINNER
RCP007 | Chicken Salad | LUNCH
RCP012 | BBQ Chicken Wings | SNACK

Advanced Feature (for 70+ students):

- **Multi-ingredient search**: Find recipes containing ALL of multiple ingredients
- Example: Search for recipes with "chicken" AND "rice"
- Boolean logic: recipes must have ALL searched ingredients

4.2 Recipe Filtering System (8%)

Implement filters based on multiple criteria:

Filter Options:

1. **By cooking time range**
    - Example: Show recipes between 15-30 minutes
    - Input: min_time and max_time
2. **By ingredient count**
    - Example: Show recipes with 5-10 ingredients
    - Useful for finding simple/complex recipes
3. **By category** (already done in Task 2, enhance here)

Requirements:

- Allow **combining filters** (AND logic)
- Example: "DINNER recipes with 5-8 ingredients taking less than 45 minutes"
- Display count of matching recipes
- Must handle case where no recipes match filters

4.3 Recipe Modification & Deletion (7%)

Implement functions to:

a) Edit Existing Recipe

- Select recipe by ID
- Choose what to modify:
    - Change name
    - Add/remove ingredients
    - Update cooking time
    - Change category
    - Modify tags
- **Must validate** all new inputs

- Confirm changes: "Recipe RCP003 updated successfully"

b) Delete Recipe

- Select recipe by ID
- **Confirm deletion** with yes/no prompt
- Remove from data structure
- Update file immediately
- Display: "Recipe RCP005 deleted successfully"

c) Duplicate Recipe

- Create copy of existing recipe with new ID
- Prompt for new name
- Useful for variations

4.4 Advanced Statistics (No complex analytics, just counting) (5%)

Generate a **Recipe Book Summary Report**:

```
========================================
      RECIPE BOOK SUMMARY
========================================
Total Recipes: 15

Recipes by Category:
  BREAKFAST:  3
  LUNCH:      4
  DINNER:     5
  DESSERT:    2
  SNACK:      1
  BEVERAGE:   0

Cooking Time Distribution:
  Quick (<30 min):     6 recipes
```

Medium (30-60 min): 5 recipes
Long (>60 min): 4 recipes

Most Used Ingredients:
1. Eggs - appears in 8 recipes
2. Flour - appears in 6 recipes
3. Milk - appears in 5 recipes

Average Ingredients per Recipe: 6.3
Largest Recipe: RCP009 with 15 ingredients
Smallest Recipe: RCP002 with 3 ingredients
==========

Requirements:

- Count and categorize recipes
- Identify most common ingredients
- Simple calculations (average, min, max)
- **NO graphs, NO complex statistics, NO data visualization**
- Just clear, formatted text output

4.5 Design & Testing Documentation (5%)

Must submit separate document covering:

a) Design Section:

- Program structure diagram (flowchart or pseudocode)
- Data structure design decisions with justification
- Function decomposition strategy
- How you ensured modularity and reusability

b) Test Plan: Create a test table:

Test ID	Test Type	Input	Expected Output	Actual Output	Pass/Fail
T001	Validation	Recipe name: "Ab"	Error message	Error message	PASS
T002	Search	Ingredient: "egg"	3 recipes found	3 recipes found	PASS
T003	File Save	Save 5 recipes	Success message	Success message	PASS

Minimum requirements:

- 15 test cases covering all major functions
- At least 3 edge case tests
- At least 2 invalid input tests
- Test both Task 1-4 features

c) Reflection (200-300 words):

- Challenges faced and how you overcame them
- Design decisions you made and why
- What you would improve with more time

4.6 Code Quality Requirements (Applies to all tasks but emphasized here)

Your code must demonstrate:

- **Proper function decomposition** (no function should exceed 30 lines)
- **Meaningful variable names** (descriptive, not x, y, data)
- **Comments** explaining complex logic
- **No code duplication** (DRY principle)
- **Consistent formatting** (indentation, spacing)
- **Error handling** throughout
- **User-friendly menu system**

Final Program Integration

Your final submission should:

1. Start with a **welcome message** and load existing recipes
2. Display a **clear main menu** with all options
3. Execute chosen options correctly
4. Handle all errors gracefully
5. Save data before exiting
6. Provide **smooth user experience** throughout

Menu Example:

```
==========================================
  DIGITAL RECIPE BOOK MANAGER
==========================================
1. Add New Recipe
2. View All Recipes
3. View Recipe by ID
4. Search by Ingredient
5. Filter Recipes
6. Edit Recipe
7. Delete Recipe
8. View Statistics
9. Save Recipes
10. Export Recipe
11. Exit
==========================================
Enter your choice (1-11):
```

Marking Criteria for Task 4 (35%)

Band	Criteria
0-30	Basic search partially works; some features attempted; statistics incomplete; minimal testing; code organization poor; many bugs
31-40	Search functional for simple cases; most features implemented; basic statistics; some test cases; code somewhat organized; several minor bugs
41-69	All required features work correctly; good search implementation; complete statistics; comprehensive testing; well-organized code; good error handling; minor issues only
70-85	Exceptional implementation; multi-ingredient search works perfectly; advanced filtering; comprehensive statistics; thorough testing; excellent code structure; flawless error handling; bonus features implemented
85+	Outstanding work showing deep understanding; elegant solutions; sophisticated search algorithms; perfect abstraction; exemplary documentation; zero bugs; professional-quality code; innovative features beyond requirements

Overall Marking Philosophy

Grade Boundaries

Score	Grade	Interpretation
0-30	Fail	Basic attempt showing some understanding; many features incomplete; program runs but with significant issues
31-40	Pass (Lenient)	Demonstrates understanding of core concepts; most features work; acceptable quality for first semester
41-55	Good Pass	Solid implementation; all main features functional; good code organization
56-69	Very Good	Strong work; robust implementation; good design decisions; comprehensive features

Score	Grade	Interpretation
70-85	Excellent (Hard to achieve)	Outstanding quality; exceptional problem-solving; professional-level code; innovative approach
85-100	Exceptional (Very rare)	Extraordinary work exceeding all expectations; demonstrates advanced understanding; publishable quality

Note: Scoring above 70 requires demonstrating **exceptional understanding and implementation quality** that goes well beyond basic requirements.

Submission Requirements

What to Submit:

1. **Python source file(s)**:
    - Main program: recipe_manager.py
    - Any additional modules you created
    - Must run without errors
2. **Design & Testing Document** (PDF):
    - Design diagrams/pseudocode
    - Test plan with results
    - Reflection
3. **Sample files**:
    - recipes.txt - with at least 5 sample recipes
    - Any exported recipe files
4. **README.txt**:
    - How to run your program
    - Any special instructions
    - Python version used
    - List of implemented features

File Naming Convention:

- StudentUOWID_recipe_manager.py
- StudentUOWID_design_testing.pdf
- StudentUOWID_recipes.txt

Important Notes

Academic Integrity

- This is an **individual assignment**
- You may discuss concepts but **code must be your own**
- Plagiarism will result in zero marks
- **Use of AI code generators must be declared and will be heavily scrutinized during viva**

VIVA VOCE EXAMINATION REQUIREMENT

Mandatory Viva Assessment Clause

CRITICAL NOTICE TO ALL STUDENTS

This coursework includes a MANDATORY VIVA VOCE (oral examination) component. Failure to attend or perform satisfactorily in the viva may result in significant mark reduction or assignment failure, regardless of submitted work quality.

Viva Voce Examination Policy

1. Mandatory Requirement

All students MUST attend a viva voce examination as part of this coursework assessment.

- The viva is **NOT optional**
- Marks awarded are **contingent upon viva performance**

- Submitted work will be provisionally marked, but **final marks are subject to viva verification**
- Students who fail to attend their scheduled viva without approved mitigating circumstances will receive **zero marks** for the entire coursework

2. Viva Examination Details

Duration

- **Minimum duration: 15 minutes**
- **Maximum duration: 30 minutes**
- Duration may be extended at examiner's discretion if clarification is needed

Format

- **One-to-one** examination with module leader/examiner
- May include additional observer for quality assurance
- Conducted in person or via video call (as specified)
- **No materials or notes permitted** unless specifically stated
- **No preparation time given** - questions begin immediately

Scheduling

- Vivas will be scheduled within **1 week of final submission deadline**
- Students will receive **minimum 3 working days' notice** of viva time/date
- **Only ONE viva attempt is permitted** - there are no resits for viva examinations
- Students must attend at the scheduled time
- Rescheduling only permitted for documented emergencies (mitigating circumstances apply)

3. Viva Examination Content

During the viva, you will be required to:

A. Demonstrate Code Ownership

- Explain your code line-by-line when requested
- Justify design decisions and implementation choices
- Describe your problem-solving approach
- Explain the purpose and function of specific code segments
- Trace program execution through your code
- Identify and explain outputs for given inputs

B. Prove Genuine Understanding

- Answer questions about programming concepts used in your code
- Explain data structures, algorithms, and logic employed
- Demonstrate understanding of Python syntax and semantics
- Modify code on-the-spot as directed by examiner
- Debug code sections presented by examiner
- Explain error messages and their solutions

C. Defend Implementation Decisions

- Justify why you chose specific approaches
- Explain alternative solutions you considered
- Discuss trade-offs in your design
- Demonstrate awareness of code limitations
- Explain your testing methodology
- Answer questions about your design documentation

4. Viva Performance Grading

Your viva performance will impact the final grade according to the following criteria:

How Viva Performance Affects Your Final Mark: Your submitted coursework will be marked out of 100 based on the quality of your code, documentation, and adherence to requirements. However, your *final* mark for the coursework will be determined by multiplying your submission mark by the percentage indicated in the viva performance table below. For

example, if your submission receives 80/100 but your viva performance is assessed as "Good", your final mark would be calculated as: 80 × 65% (midpoint of 60-69% range) = 52/100. This system ensures that marks reflect both the quality of submitted work and demonstrated understanding.

Performance Level	**Description**	**Impact on Final Mark**
Excellent	Demonstrates complete ownership; answers all questions confidently and accurately; shows deep understanding; can modify code competently	**70-100% of submission marks awarded**
Good	Demonstrates clear ownership; answers most questions correctly; shows solid understanding; minor hesitations acceptable	**60-69% of submission marks awarded**
Satisfactory	Demonstrates basic ownership; answers key questions; shows adequate understanding; some gaps acceptable	**50-59% of submission marks awarded**
Weak	Struggles to explain own code; significant gaps in understanding; cannot answer basic questions	**40-49% of submission marks awarded**
Poor	Cannot explain code; minimal understanding demonstrated; extensive prompting required	**30-39% of submission marks awarded**
Failed	Cannot demonstrate ownership; unable to explain basic concepts; clear evidence	**0-29% of submission marks awarded**

Questions?

Contact your module tutor via the module discussion board designated for this coursework.

Good luck, and happy coding!

DETAILED MARKING GUIDE

Digital Recipe Book Manager

Total Marks: 100

TASK 1 – Recipe Input & Validation (20 Marks)

1.1 Recipe Name Validation (5 marks)

Criterion	Marks	Details
Length validation (3-50 characters)	1.0	Correctly rejects names < 3 or > 50 characters
Contains at least one letter check	0.5	Rejects purely numeric inputs (e.g., "123")
Empty/whitespace-only rejection	0.5	Rejects empty strings and strings with only spaces
Special character validation	1.0	Allows only hyphen, apostrophe, space; rejects other special chars
Case handling	0.5	Accepts input in any case (converts or accepts as-is consistently)
Re-prompting on invalid input	1.0	Continuously prompts until valid name is entered
Clear error messages	0.5	Displays specific error for each validation failure

Subtotal: 5.0 marks

1.2 Ingredient Input Validation (6 marks)

Criterion	Marks	Details
Minimum ingredient count (3)	0.5	Enforces at least 3 ingredients
Maximum ingredient count (20)	0.5	Enforces no more than 20 ingredients
Ingredient name validation (2-30 chars)	1.0	Checks length for each ingredient name
Quantity validation - positive number	1.0	Rejects zero, negative, and non-numeric quantities
Quantity handles decimals	0.5	Accepts both integer and decimal quantities (e.g., 2.5)
Unit validation from allowed list	1.0	Only accepts: g, kg, ml, l, cup, tbsp, tsp, piece
Unit case-insensitive	0.5	Accepts "G", "g", "ML", "ml" etc.
Duplicate ingredient detection	0.5	Identifies when same ingredient entered twice
Duplicate handling prompt	0.5	Asks user to confirm or reject duplicate
Input format parsing	0.5	Correctly splits ingredient string (e.g., "Flour, 250, g")

Subtotal: 6.0 marks

1.3 Cooking Time Validation (3 marks)

Criterion	Marks	Details
Format validation (HH:MM)	0.5	Checks for colon separator and correct format
Minimum time (00:05)	0.5	Rejects times less than 5 minutes
Maximum time (12:00)	0.5	Rejects times greater than 12 hours

Criterion	Marks	Details
Hours validation (0-12)	0.5	Ensures hours are in valid range
Minutes validation (0-59)	0.5	Ensures minutes are in valid range (rejects 60+)
Re-prompting on invalid time	0.5	Continues to prompt until valid time entered

Subtotal: 3.0 marks

1.4 Category Selection (2.5 marks)

Criterion	Marks	Details
Displays all valid categories	0.5	Shows complete list: BREAKFAST, LUNCH, DINNER, DESSERT, SNACK, BEVERAGE
Accepts only valid categories	0.5	Rejects any input not in the predefined list
Case-insensitive input	0.5	Accepts "breakfast", "BREAKFAST", "Breakfast" etc.
Re-prompting on invalid category	0.5	Continuously prompts until valid selection made
Clear error message	0.5	States what went wrong and shows valid options

Subtotal: 2.5 marks

1.5 Function Implementation & Output (3.5 marks)

Criterion	Marks	Details
validate_recipe_input() function exists	0.5	Function is properly defined

Criterion	Marks	Details
Function collects all required components	0.5	Gathers name, ingredients, time, and category
Function returns validated data	0.5	Returns structured data (dict/tuple) not just prints
Validation summary display	1.0	Shows formatted summary with all validated details
Summary formatting quality	0.5	Clear, readable format with proper alignment
Success indicator present	0.5	Shows checkmark or "Successfully Validated" message

Subtotal: 3.5 marks

TASK 1 TOTAL: 20.0 marks

TASK 2 – Recipe Storage & Display (20 Marks)

2.1 Data Structure Design (5 marks)

Criterion	Marks	Details
Dictionary as primary container	1.0	Uses dictionary with recipe ID as key
Nested structure for recipe details	1.0	Each recipe is a dictionary/object with multiple fields
List used for ingredients storage	0.5	Ingredients stored in a list
Tuple used for ingredient details	1.0	Each ingredient is a tuple: (name, quantity, unit)
Set used for tags/categories	1.0	Uses set data structure for unique tags
Structure is logical and accessible	0.5	Easy to retrieve and manipulate data

Subtotal: 5.0 marks

2.2 Recipe ID Generation (2 marks)

Criterion	Marks	Details
Correct format (RCP + 3 digits)	0.5	IDs follow RCP001, RCP002 format
Starts from RCP001	0.5	First recipe gets RCP001
Auto-incrementing logic	0.5	Each new recipe gets next number
No duplicate IDs	0.5	Ensures uniqueness even after deletions

Subtotal: 2.0 marks

2.3 Multiple Recipe Entry (2.5 marks)

Criterion	Marks	Details
Allows adding multiple recipes	0.5	Loop structure permits multiple entries
"Add another?" prompt	0.5	Asks user after each recipe
Accepts yes/no input	0.5	Handles affirmative and negative responses
All recipes stored correctly	0.5	All entered recipes persist in data structure
Recipe count tracked	0.5	Maintains count of total recipes

Subtotal: 2.5 marks

2.4 Display Functions (7 marks)

a) Display Single Recipe - Detailed (2.5 marks)

Criterion	Marks	Details
Shows recipe ID	0.5	Correctly displays unique identifier
Shows name, category, time	0.5	All basic details visible
Ingredients formatted as numbered list	0.5	Each ingredient on separate line with number
Ingredient details formatted correctly	0.5	Shows name - quantity unit format
Tags displayed	0.5	Shows all tags for the recipe

b) Display All Recipes - Summary (2.5 marks)

Criterion	Marks	Details
Shows total recipe count	0.5	Displays "Total Recipes: X"
Lists all recipes	0.5	Every recipe in collection appears
Summary format correct	0.5	ID

Criterion	Marks	Details
Aligned columns	0.5	Uses consistent spacing/alignment
Handles empty collection	0.5	Displays appropriate message if no recipes

c) Display by Category (2.0 marks)

Criterion	Marks	Details
Prompts for category selection	0.5	Asks user which category to filter
Filters recipes correctly	0.5	Shows only matching category recipes
Shows count of matching recipes	0.5	Displays "Found X recipes"
Handles no matches	0.5	Appropriate message if category is empty

Subtotal: 7.0 marks

2.5 Recipe Operations (3.5 marks)

Criterion	Marks	Details
Add recipe function	0.5	Separate function to add recipe to collection
View specific recipe by ID	0.5	Can retrieve and display single recipe
List all recipe IDs with names	0.5	Shows ID-name pairs for all recipes
Count recipes per category	0.5	Can calculate and display category counts
Menu-driven interface exists	0.5	Numbered menu with clear options
Menu choice validation	0.5	Rejects invalid menu selections
Graceful handling of empty book	0.5	Appropriate messages when no recipes exist
Clear feedback for all operations	0.5	Success/failure messages for each action

Subtotal: 3.5 marks

TASK 2 TOTAL: 20.0 marks

TASK 3 – File Management & Persistence (25 Marks)

3.1 Save Recipes to File (7.5 marks)

Criterion	Marks	Details
Creates/opens recipes.txt file	0.5	File is created in correct location
Structured format implementation	1.5	Uses delimited format (===RECIPE===, fields, ===END===)
All recipe fields saved	1.0	ID, name, category, time, ingredients, tags all written
Ingredients format correct	1.0	Multiple ingredients properly delimited (e.g., pipe or semicolon)
Tags format correct	0.5	Tags saved in parseable format
File writing error handling	1.0	Try-except block catches file write errors
Success confirmation message	0.5	Displays "Recipes saved successfully"
Preserves existing recipes	0.5	Doesn't lose data on multiple saves
Triggered by menu option	0.5	User can choose when to save

Subtotal: 7.5 marks

3.2 Load Recipes from File (8 marks)

Criterion	Marks	Details
Checks if file exists	0.5	Uses file existence check before attempting to open
Handles missing file gracefully	0.5	Displays "No saved recipes" message, doesn't crash
Opens and reads file correctly	0.5	Successfully reads recipes.txt

Criterion	Marks	Details
Parses recipe delimiters	1.0	Correctly identifies ===RECIPE=== and ===END=== markers
Parses each field correctly	1.5	Extracts ID, name, category, time from file
Reconstructs ingredient list	1.0	Parses ingredient string back into list of tuples
Reconstructs tags set	0.5	Converts tag string back to set
Data validation on load	1.0	Validates loaded data before adding to program
Corrupted file handling	1.0	Catches parsing errors, displays specific error message
Success message with count	0.5	Shows "Loaded X recipes successfully"

Subtotal: 8.0 marks

3.3 Auto-save Feature (3 marks)

Criterion	Marks	Details
Auto-save implemented	1.0	One of: after each add, before exit, or periodic
Auto-save logic correct	1.0	Triggers at appropriate times
User notification	0.5	Indicates when auto-save occurs
Doesn't interfere with flow	0.5	Doesn't disrupt user experience

Subtotal: 3.0 marks

3.4 Export Single Recipe (3 marks - Bonus within Task 3)

Criterion	Marks	Details
Creates separate file per recipe	0.5	Filename format: recipe_<name>.txt

Criterion	Marks	Details
Human-readable formatting	1.0	Clear, formatted layout not machine format
All recipe details included	0.5	Name, category, time, ingredients, tags present
Export function works correctly	0.5	Successfully creates and writes file
Handles filename special characters	0.5	Sanitizes recipe name for valid filename

Subtotal: 3.0 marks

3.5 File Operation Menu Integration (1.5 marks)

Criterion	Marks	Details
Save option in menu	0.5	Menu includes "Save to File"
Load option in menu	0.5	Menu includes "Load from File" (or auto-loads)
Export option in menu	0.5	Menu includes "Export Recipe"

Subtotal: 1.5 marks

3.6 Error Handling (2 marks)

Criterion	Marks	Details
FileNotFoundError handled	0.5	Catches and handles missing file
PermissionError handled	0.5	Catches and handles permission issues
Corrupted data handling	0.5	Detects and handles malformed file content
Empty file handling	0.5	Handles empty recipes.txt appropriately

Subtotal: 2.0 marks

TASK 3 TOTAL: 25.0 marks

TASK 4 – Search & Advanced Features (35 Marks)

4.1 Ingredient-Based Search (10 marks)

Basic Search (6 marks)

Criterion	Marks	Details
Search function exists	0.5	Dedicated search function implemented
Prompts for ingredient name	0.5	Asks user what ingredient to search for
Case-insensitive search	1.0	Finds "Chicken", "chicken", "CHICKEN"
Partial match capability	1.0	"chick" finds "chicken", "chick peas"
Searches all recipes	0.5	Iterates through entire recipe collection
Returns all matching recipes	1.0	Finds every recipe containing the ingredient
Display format correct	1.0	Shows count and formatted list (ID
Handles no matches	0.5	Displays "No recipes found with..." message

Subtotal: 6.0 marks

Multi-Ingredient Search (4 marks - Advanced)

Criterion	Marks	Details
Accepts multiple ingredients	1.0	User can enter 2+ ingredients to search
AND logic implementation	1.5	Recipe must contain ALL searched ingredients
Correctly filters results	1.0	Only shows recipes with all ingredients
Clear user instructions	0.5	Explains how to enter multiple ingredients

Subtotal: 4.0 marks

Section Total: 10.0 marks

4.2 Recipe Filtering System (8 marks)

Filter by Cooking Time (3 marks)

Criterion	Marks	Details
Accepts min and max time	0.5	Prompts for both minimum and maximum
Time range validation	0.5	Ensures min ≤ max
Filters recipes correctly	1.0	Returns only recipes within time range
Time comparison logic correct	1.0	Properly converts HH:MM and compares

Subtotal: 3.0 marks

Filter by Ingredient Count (2.5 marks)

Criterion	Marks	Details
Accepts min and max count	0.5	Prompts for ingredient count range
Range validation	0.5	Ensures min ≤ max, both positive
Counts ingredients correctly	0.5	Accurately counts ingredients in each recipe
Filters correctly	1.0	Shows recipes within ingredient count range

Subtotal: 2.5 marks

Combined Filters (2.5 marks)

Criterion	Marks	Details
Can apply multiple filters together	1.0	Allows time + count + category filtering
AND logic for combined filters	1.0	Recipe must match ALL active filters
Displays combined results correctly	0.5	Shows count and recipes matching all criteria

Subtotal: 2.5 marks

Section Total: 8.0 marks

4.3 Recipe Modification & Deletion (7 marks)

Edit Recipe (4 marks)

Criterion	Marks	Details
Select recipe by ID	0.5	Prompts for recipe ID to edit
Validates recipe ID exists	0.5	Checks if ID is in collection
Shows edit menu options	0.5	Lists: name, ingredients, time, category, tags
Allows changing name	0.5	Can update recipe name
Allows modifying ingredients	0.5	Can add/remove/change ingredients
Allows updating time	0.5	Can change cooking time
Validates all new inputs	0.5	Re-uses validation functions for edits
Confirmation message	0.5	Shows "Recipe updated successfully"

Subtotal: 4.0 marks

Delete Recipe (2 marks)

Criterion	Marks	Details
Select recipe by ID	0.5	Prompts for recipe ID to delete
Confirmation prompt	0.5	Asks "Are you sure?" before deleting
Removes from data structure	0.5	Recipe deleted from main collection
Success message	0.5	Shows "Recipe deleted successfully"

Subtotal: 2.0 marks

Duplicate Recipe (1 mark - Bonus within section)

Criterion	Marks	Details
Creates copy with new ID	0.5	New recipe ID generated
Prompts for new name	0.5	Asks user to rename the duplicate

Subtotal: 1.0 mark

Section Total: 7.0 marks

4.4 Recipe Book Summary Statistics (5 marks)

Criterion	Marks	Details
Total recipe count	0.5	Shows "Total Recipes: X"
Recipes by category breakdown	1.0	Lists count for each category
Cooking time distribution	1.0	Categorizes into Quick/Medium/Long with counts
Most used ingredients	1.5	Identifies top 3 ingredients with occurrence count
Average ingredients calculation	0.5	Calculates and displays average correctly
Largest recipe identified	0.5	Shows recipe with most ingredients
Smallest recipe identified	0.5	Shows recipe with fewest ingredients
Report formatting	0.5	Clean, readable, well-structured output

Section Total: 5.0 marks

4.5 Design & Testing Documentation (5 marks)

Design Section (2 marks)

Criterion	Marks	Details
Program structure diagram	0.5	Flowchart or pseudocode showing program flow
Data structure explanation	0.5	Justifies choice of dictionaries, lists, tuples, sets
Function decomposition strategy	0.5	Explains how program is broken into functions
Modularity discussion	0.5	Describes how code is organized for reusability

Subtotal: 2.0 marks

Test Plan (2.5 marks)

Criterion	Marks	Details
Test table format	0.5	Uses proper table with all required columns
Minimum 15 test cases	1.0	At least 15 distinct tests documented
Edge case tests (min 3)	0.5	Tests boundary conditions
Invalid input tests (min 2)	0.5	Tests error handling

Subtotal: 2.5 marks

Reflection (0.5 marks)

Criterion	Marks	Details
Challenges and solutions discussed	0.5	200-300 words reflecting on experience

Subtotal: 0.5 marks

Section Total: 5.0 marks

TASK 4 TOTAL: 35.0 marks

OVERALL CODE QUALITY (Distributed across all tasks)

Note: These marks are already embedded in the criteria above but summarized here for clarity:

Function Decomposition

- Functions under 30 lines: included in implementation marks
- No code duplication: included in quality marks
- Meaningful function names: included throughout

Variable Naming & Comments

- Descriptive variable names: assessed in each task
- Comments explaining logic: assessed in quality criteria
- Consistent formatting: assessed in display/output marks

Error Handling

- Try-except blocks: explicitly marked in file handling and validation
- Graceful error messages: marked in validation sections
- User-friendly prompts: marked in menu and input sections

User Experience

- Clear menu system: marked in Task 2 and Task 4
- Smooth program flow: marked in integration criteria
- Welcome/exit messages: marked in final integration